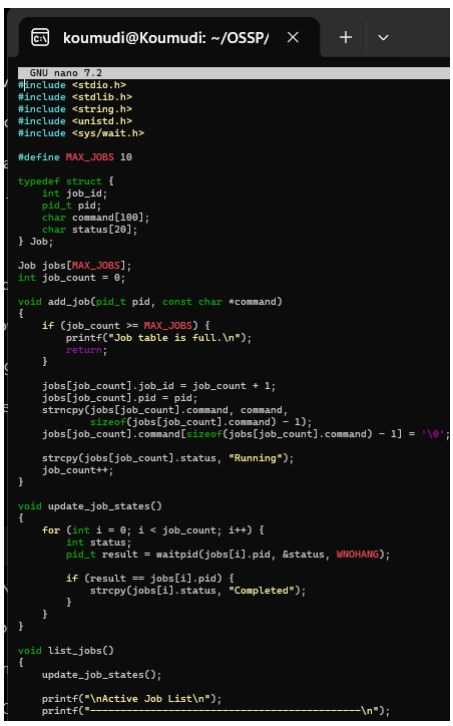
Skill OSSP

Week 9 (S 17 & 18)

SESSION – 17:

TASK1: To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.



```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct {
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    strncpy(jobs[job_count].command, command,
            sizeof(jobs[job_count].command) - 1);
    jobs[job_count].command[sizeof(jobs[job_count].command) - 1] = '\0';

    strcpy(jobs[job_count].status, "Running");
    job_count++;
}

void update_job_states()
{
    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result == jobs[i].pid) {
            strcpy(jobs[i].status, "Completed");
        }
    }
}

void list_jobs()
{
    update_job_states();

    printf("\nActive Job List\n");
    printf("-----------------------------------------------\n");
```



```
GNU nano 7.2
    printf("Job ID\tPID\tStatus\t\tCommand\n");
    printf("-----------------------------------------------\n");

    for (int i = 0; i < job_count; i++) {
        printf("[%d]\t%d\t%-10s\t%s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].status,
               jobs[i].command);
    }

    printf("-----------------------------------------------\n");
}

int main()
{
    pid_t pid1, pid2;

    printf("Creating background jobs...\n");

    pid1 = fork();

    if (pid1 == 0) {
        sleep(5);
        exit(0);
    }

    add_job(pid1, "sleep 5");

    pid2 = fork();

    if (pid2 == 0) {
        sleep(8);
        exit(0);
    }

    add_job(pid2, "sleep 8");

    list_jobs();

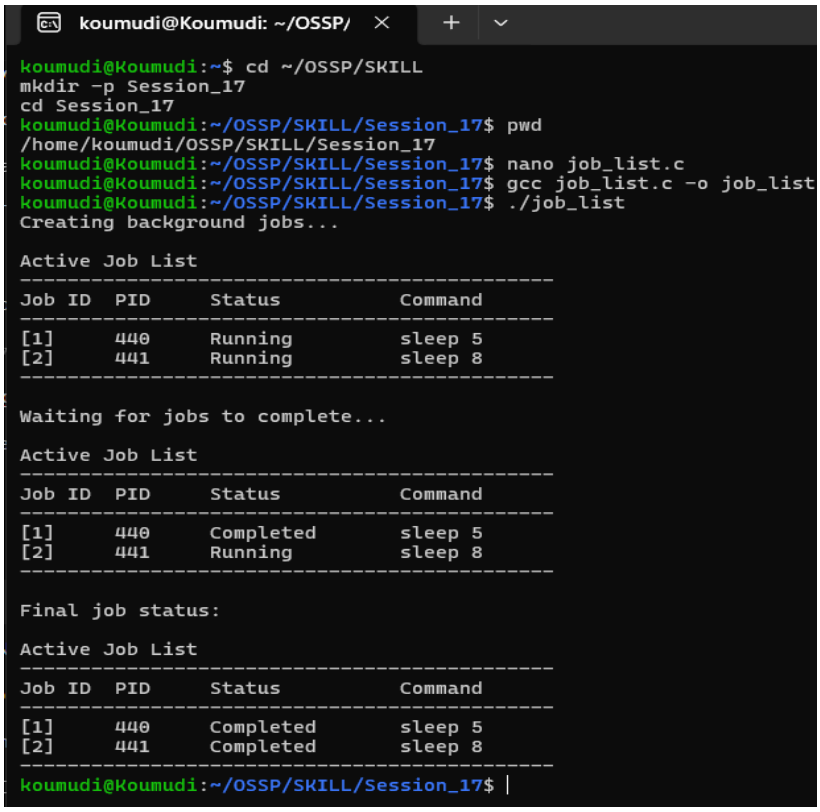
    printf("\nWaiting for jobs to complete...\n");
    sleep(6);

    list_jobs();

    printf("\nFinal job status:\n");
    sleep(3);

    list_jobs();

    return 0;
}
```



```
koumudi@Koumudi:~$ cd ~/OSSP/SKILL
mkdir -p Session_17
cd Session_17
koumudi@Koumudi:~/OSSP/SKILL/Session_17$ pwd
/home/koumudi/OSSP/SKILL/Session_17
koumudi@Koumudi:~/OSSP/SKILL/Session_17$ nano job_list.c
koumudi@Koumudi:~/OSSP/SKILL/Session_17$ gcc job_list.c -o job_list
koumudi@Koumudi:~/OSSP/SKILL/Session_17$ ./job_list
Creating background jobs...

Active Job List
-----------------------------------------------
Job ID  PID     Status          Command
-----------------------------------------------
[1]     440     Running         sleep 5
[2]     441     Running         sleep 8
-----------------------------------------------

Waiting for jobs to complete...

Active Job List
-----------------------------------------------
Job ID  PID     Status          Command
-----------------------------------------------
[1]     440     Completed       sleep 5
[2]     441     Running         sleep 8
-----------------------------------------------

Final job status:

Active Job List
-----------------------------------------------
Job ID  PID     Status          Command
-----------------------------------------------
[1]     440     Completed       sleep 5
[2]     441     Completed       sleep 8
-----------------------------------------------
koumudi@Koumudi:~/OSSP/SKILL/Session_17$
```

TASK2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

int main()
{
    pid_t pid;
    int status;

    printf("Starting a background job...\n");

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started. PID = %d\n", getpid());

        sleep(2);

        printf("Child process completed.\n");
        exit(0);
    }

    printf("Parent: Background job created.\n");
    printf("Parent: Job PID = %d\n", pid);

    sleep(1);

    printf("\nSwitching job to foreground...\n");
    printf("Parent: Waiting for foreground job to complete...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Foreground job completed successfully.\n");
    }

    printf("Parent: Foreground switching completed.\n");

    return 0;
}
```

```
koumudi@Koumudi:~/OSSP/SKILL/Session_17$ nano foreground.c
koumudi@Koumudi:~/OSSP/SKILL/Session_17$ gcc foreground.c -o foreground
koumudi@Koumudi:~/OSSP/SKILL/Session_17$ ./foreground
Starting a background job...
Parent: Background job created.
Parent: Job PID = 461
Child process started. PID = 461

Switching job to foreground...
Parent: Waiting for foreground job to complete...
Child process completed.
Foreground job completed successfully.
Parent: Foreground switching completed.
koumudi@Koumudi:~/OSSP/SKILL/Session_17$
```

SESSION – 18:

TASK1: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

koumudi@Koumudi: ~/OSSP/

GNU nano 7.2

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handle_sigint(int sig)
{
    printf("\nChild: SIGINT received.\n");
}

void handle_sigterm(int sig)
{
    printf("Child: SIGTERM received.\n");
}

int main()
{
    pid_t pid;
    sigset_t mask;

    printf("Parent PID: %d\n", getpid());
    printf("Parent Process Group ID: %d\n", getpgrp());

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild PID: %d\n", getpid());
        printf("Child Process Group ID: %d\n", getpgrp());

        signal(SIGINT, handle_sigint);
        signal(SIGTERM, handle_sigterm);

        sigemptyset(&mask);
        sigaddset(&mask, SIGINT);

        sigprocmask(SIG_BLOCK, &mask, NULL);

        printf("Child: SIGINT is blocked for 5 seconds.\n");
        sleep(5);

        sigprocmask(SIG_UNBLOCK, &mask, NULL);

        printf("Child: SIGINT is unblocked.\n");
        printf("Child: Waiting for signals...\n");

        sleep(5);
        printf("Child: Waiting for signals...\n");

        sleep(5);

        printf("Child: Exiting.\n");
        exit(0);
    }

    printf("\nParent: Child PID = %d\n", pid);
    printf("Parent: Sending SIGTERM to child...\n");

    sleep(2);
    kill(pid, SIGTERM);

    waitpid(pid, NULL, 0);

    printf("\nParent: Child process completed.\n");

    return 0;
}
```

```
koumudi@Koumudi:~/OSSP/SKILL/Session_17$ cd ~/OSSP/SKILL
koumudi@Koumudi:~/OSSP/SKILL$ mkdir -p Session_18
cd Session_18
nano signal_analysis.c
koumudi@Koumudi:~/OSSP/SKILL/Session_18$ gcc signal_analysis.c -o signal_analysis
koumudi@Koumudi:~/OSSP/SKILL/Session_18$ ./signal_analysis
Parent PID: 598
Parent Process Group ID: 598

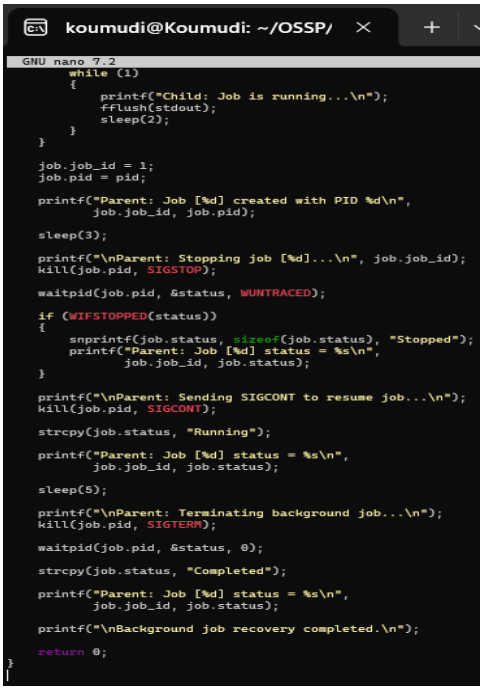
Parent: Child PID = 599
Parent: Sending SIGTERM to child...

Child PID: 599
Child Process Group ID: 598
Child: SIGINT is blocked for 5 seconds.
Child: SIGTERM received.
Child: SIGINT is unblocked.
Child: Waiting for signals...
Child: Exiting.

Parent: Child process completed.
koumudi@Koumudi:~/OSSP/SKILL/Session_18$
```

TASK2: To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.






```
koumudi@Koumudi:~/OSSP/SKILL/Session_18$ nano background_recovery.c
koumudi@Koumudi:~/OSSP/SKILL/Session_18$ gcc background_recovery.c -o background_recovery
koumudi@Koumudi:~/OSSP/SKILL/Session_18$ ./background_recovery
Parent: Job [1] created with PID 642
Child: Job started. PID = 642
Child: Job is running...
Child: Job is running...

Parent: Stopping job [1]...
Parent: Job [1] status = Stopped

Parent: Sending SIGCONT to resume job...
Parent: Job [1] status = Running
Child: Job is running...
Child: Job is running...

Parent: Terminating background job...
Parent: Job [1] status = Completed

Background job recovery completed.
koumudi@Koumudi:~/OSSP/SKILL/Session_18$
```